

BoxD: Managing GPU Managed Memory

Anonymous Author(s)

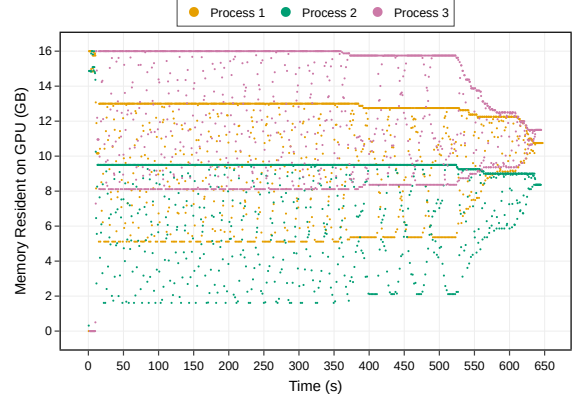
Abstract

Performance-sensitive GPU applications require deterministic resource provisioning. NVIDIA’s Unified Virtual Memory (UVM) enables GPU memory oversubscription but relies on a device-centric, application-agnostic eviction mechanisms. While oversubscription improves co-location, global device-level evictions induce interference and non-deterministic memory provisioning. We demonstrate the severity of this effect with different oversubscription scenarios by quantifying the mean and p99 runtime inflation, increase in evictions and page fault amplification. Static compute and memory provisioning techniques do not preserve working-set stability or service-level differentiation. We present **BoxD**, a cgroup-backed mechanism that integrates GPU-managed memory into the Linux resource control framework. BoxD enforces hard and soft limits on managed memory residency, supports custom per-cgroup eviction policies, and enables deterministic multi-tenant GPU memory provisioning without modifying UVM’s core allocation mechanisms. Our evaluation, with a tiered isolation setup, shows that BoxD reduces the volume of eviction by up to 50 $\times$ and increases throughput by 1.5 $\times$ in a tier-based isolation setup, and preserves service-level performance targets.

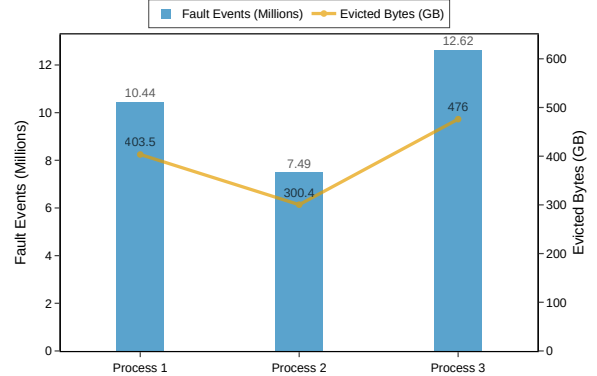
1 Introduction

Modern high-performance workloads increasingly rely on GPUs to accelerate data-intensive applications, from graph analytics to large language model serving[17]. To improve utilization and amortize hardware costs, GPUs are frequently shared across multiple concurrent processes. Compute multiplexing mechanisms such as time slicing [7], MPS [25], and MIG [24] provide varying degrees of scheduling and isolation. Although these techniques have been used for elastic compute provisioning [15, 31], GPU memory management remains fundamentally device-centric and driver-controlled. In particular, NVIDIA’s Unified Virtual Memory (UVM) [28] abstraction, which enables oversubscription by transparently migrating memory pages between CPU and GPU on demand. This abstraction is referred to as **managed memory**, the UVM driver handling the management of memory mappings on demand, transparent to the process. UVM simplifies programming and allows workloads to exceed physical GPU memory capacity. However, GPU memory residency and eviction decisions are implemented within the driver using a device-level view of GPU memory without process-level or application-level accounting, provisioning, and isolation. This design has a critical consequence — *with oversubscription, GPU memory residency is non-deterministic and sensitive to device-level global eviction ordering rather than individual workload semantics and requirements*.

To illustrate the non-determinism of per-process GPU resident memory, we conduct an experiment with three identical GPU processes. Each process allocates 16 GB of managed memory using UVM and accesses memory in a uniform stride pattern, touching every 128th byte. All processes execute concurrently on an NVIDIA Blackwell 4500 (32GB) GPU, resulting in memory oversubscription



(a) Resident memory on the GPU over time.



(b) GPU memory multiplexing of co-located processes.

Figure 1: Non-deterministic GPU memory residency and eviction amplification with oversubscribed UVM memory.

(48 GB total demand). Figure 1a shows the resident memory of each process over time. Despite the identical allocation size, identical access patterns, and identical execution configuration, the observed residency diverges significantly between processes. The resident memory oscillates sharply, from 16 GB to 2 GB, across all processes. These differences arise solely from UVM’s device-specific and process-agnostic policies. This behavior resembles steady-state thrashing, in which working sets are repeatedly displaced and reloaded without any notion of reservation, proportion, or priority. This instability is also reflected in the fault and eviction statistics. As shown in Figure 1b, each process incurs between 7.5 million to 12.6 million page fault events and triggers 300GB to 475GB of cumulative GPU memory evictions during execution. Although each process only allocates 16GB, the driver migrates hundreds of GBs of data due to repeated eviction-refault cycles.

Table 1: Comparison of memory allocation methodologies for NVIDIA GPU architectures.

Type	Runtime Allocation	Scope	Transfer	Migratable	Prone to interference	Oversubscription	IPC Available
Global	cudaMalloc()	Single GPU	cudaMemcpy()	✗	✗	✗	✓
Pinned	cudaMallocHost()	Host	cudaMemcpy()	✗	✗	✗	✗
Shared	__shared__	Thread Block	Copy from Global	NA	✗	✗	NA
Unified	cudaMallocManaged()	Host & Device	On-demand	✓	✓	✓	✗
Heterogeneous	malloc()	Host & Device	On-demand	✓	✓	✓	✗

The lack of process-aware GPU memory control presents a fundamental mismatch with modern multi-tenant and containerized systems. The operating system provides well-defined abstractions for managing CPU time, DRAM, and I/O resources, yet GPU-managed memory remains outside this governance framework. Under contention, eviction decisions are driven by opaque driver heuristics rather than explicit policy, leading to interference, non-deterministic provisioning, leading to unpredictable performance.

As part of this work, we address this gap by introducing **BoxD**, a cgroup-backed mechanism that integrates GPU managed memory into the Linux resource control subsystem. By embedding UVM memory management within the kernel’s cgroup abstraction, BoxD replaces opaque global eviction with explicit, policy-driven control, restoring predictability to GPU memory provisioning under oversubscription. BoxD exposes GPU-resident memory as a first-class, OS-managed resource and enables:

- Per-cgroup soft and hard limits on GPU residency
- Deterministic bounds on cross-process interference
- Cgroup-aware eviction policies, configurable at runtime
- Per-GPU accounting in multi-GPU systems

Using the above features, **BoxD** enables per-cgroup deterministic GPU memory guarantees, cgroup-native eviction policies and memory management determinism with multi-GPU setups. As an example, we repeated the experiment described in Figure 1, with differentiated memory provisioning, and could enable a prioritized process setup that experienced no evictions and page faults proportional to its work set size and access pattern. As part of this work, we make the following contributions:

- Characterize the internal behavior of UVM-based memory allocations, evictions under oversubscription, and demonstrate non-deterministic residency and eviction amplification.
- Design and implement **BoxD**, a cgroup-integrated GPU memory control mechanism that introduces process-aware limits and eviction policies.
- An extensive evaluation of **BoxD**’s correctness, feature set—deterministic and dynamic provisioning and custom policies, and applicability with multi-GPU platforms.

The remainder of the paper is organized as follows. Section 2 provides background on GPU memory allocation models and UVM internals. Section 3 characterizes the interference problem in detail. Section 4 goes in depth about the design philosophy and implementation details of **BoxD**. Section 5 evaluates the correctness of BoxD and the impact of BoxD on performance, with synthetic and real-world workloads.

2 Background

In this section, we discuss the different techniques to allocate and control GPU memory residency, from the perspective of processes.

2.1 Memory Allocation Models

NVIDIA GPUs expose multiple APIs for memory allocation with different semantics, as summarized in Table 1. The primary differentiator is the scope of the allocation, i.e., the set of residency locations available to a process.

Traditional *Global Memory* (via `cudaMalloc`) is resident on a single GPU and requires explicit programmer orchestration (e.g., `cudaMemcpy`) for all data movement. While such global memory natively supports Inter-Process Communication (IPC) and is isolated from activity in other processes, it cannot be transparently migrated between GPU memory and CPU DRAM or be oversubscribed. Similarly, *Pinned Host Memory* allocations reside in the system DRAM to improve transfer performance but incur high latency, as all device accesses must traverse the PCIe interconnect fabric [6]. To simplify these complex data paths for AI workloads, higher-level abstractions such as NIXL [22, 29] provide a modular plug-in architecture for managing various memory types. [It is an inference-pipeline aware data orchestrator, providing data through the fastest transport means at the precise time.](#)

Unified Virtual Memory (UVM) and Heterogeneous Memory Management (HMM) [9] overcome these limitations by providing a unified virtual address space common to both the CPU and the GPU. Unlike explicit memory management models, UVM follows an **on-demand paging** model where memory is instantiated lazily on the faulting processor. This decoupling of allocation from placement allows residency to be driven by dynamic access patterns and rate of accesses, enabling GPU memory **oversubscription**. However, as shown in Table 1, this flexibility makes UVM prone to interference as performance becomes dependent on the driver’s ability to manage GPU memory under contention.

2.2 UVM Driver Details

As part of CUDA initialization in a process, the CUDA driver creates a virtual address space per-process, called `uvm_va_space` (hereafter `va_space`). `va_space` creations happen irrespective of whether the process uses managed memory (UVM or HMM) or not. The `va_space` persists throughout the lifetime of the process and serves as the primary representation on the driver-side, tracking all UVM-related states and properties. Upon a memory allocation, the driver creates a `uvm_va_range` (hereafter `va_range`) to represent the allocated virtual address interval. As shown in Figure 2, `va_ranges` is owned by the `va_space` of a process. Each `va_range` subdivided into smaller, fixed-size addressable units called `uvm_va_blocks`

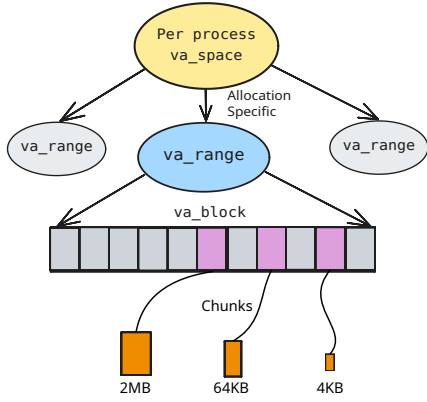


Figure 2: Per-process memory hierarchy in the UVM model.

(hereafter *va_blocks*). In contrast, under HMM, the driver does not create *va_ranges* instead, *va_blocks* are attached directly to the *va_space*. The *va_block* forms the lowest level of the UVM hierarchy. The block tracks where does it currently reside, the CPU or the GPU. The virtual-to-physical mappings for the current resident device is also stored in the *va_block*. The physical backing of a *va_block* is provided by memory chunks, whose sizes vary depending on allocation and hardware capabilities. The largest chunk size supported across NVIDIA GPU architectures is of size 2MB. When a device accesses a virtual address that is not resident in the device accessible memory, a page fault is generated. When a fault occurs, the driver receives the fault information and checks if the fault is valid [14]. For a valid fault, the driver instantiates Physical Memory Manager (PMM) for each GPU. The PMM is responsible for tracking and managing the mapping physical memory chunk. Figure 2 illustrates the memory-management components within the UVM driver. These components track GPU-specific metadata, including GPU memory mappings and residency information. If a process uses multiple GPUs, the driver additionally creates a virtual address space per-GPU.

The physical memory manager (PMM) maintains the set of all memory chunks available on a GPU. Figure 3 illustrates the fault-servicing pipeline. For clarity, the figure omits several auxiliary (but essential) steps, such as chunk splitting, device mask updates, NUMA-node selection, and related bookkeeping. The fault-service path when the GPU raises the fault is as follows: ① A page fault is raised by the GPU and delivered to the driver [14], corresponding GPU-specific bottom-half handler is invoked, while multiple GPUs may share the same driver instance, each GPU is consistently serviced by its assigned bottom-half. ② For each managed-memory fault, the driver identifies the offending *va_space* associated with the faulting process. ③ The driver checks whether the faulting address is supported by an existing *va_block*. If not, a new *va_block* is created within the corresponding *va_range*. Figure 3 assumes that the faulting address is valid; invalid accesses are rejected earlier and result in an error. ④ Once the *va_block* is established, the driver begins to populate its physical backing. As illustrated in Figure 3, the PMM attempts allocation in two phases: first without eviction and subsequently with eviction if necessary. ⑤ During

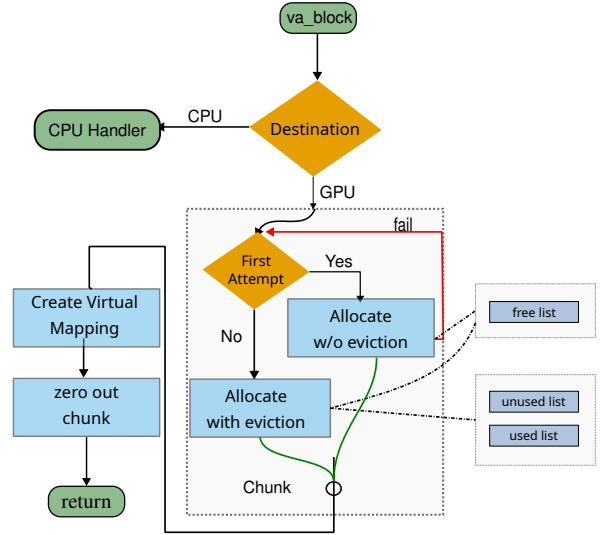


Figure 3: UVM driver's page-fault handling sequence.

eviction, the PMM first looks for free chunks. If none are available, it prioritizes eviction of unused chunks, i.e., chunks that are allocated but currently unmapped. Evicting such chunks incurs minimal overhead, as no mappings need to be invalidated. If no unused chunks are found, the PMM proceeds to evict chunks that are mapped to GPU memory. The evicted chunk is transferred to a chunk on the CPU DRAM, and the corresponding *va_block* mapping is updated to reflect residency. ⑥ After a chunk is allocated, the driver installs virtual-to-physical mappings for the fault address range. ⑦ The allocated chunk is zero-initialized to prevent exposure of residual data from previous use. ⑧ Finally, the required data is copied into the chunk, completing the fault service. At this point, the fault is resolved and the execution resumes.

Next, we discuss how the eviction strategy and the lack of control knobs lead to inefficient sharing when utilizing managed memory.

3 Problem Description

Modern high-performance workloads increasingly execute on the GPU, yet GPU memory management remains fundamentally outside the operating system's control. Allocation, residency, and eviction decisions are implemented inside the device drivers, which operate largely independently of the kernel's global resource management framework. As a result, the OS has no visibility into how much GPU memory a process or a group of processes is consuming, nor has the authority to enforce limits, perform accounting, or arbitrate contention. When multiple applications concurrently use GPU memory, especially with UVM, conflicts are resolved through opaque driver-level eviction mechanisms rather than explicit or symbiotic OS policies. This design bypasses the kernel's established abstractions for isolation and fairness, leading to interference, and highly unpredictable performance. In multi-tenant and containerized environments, where the OS is expected to provide strict resource governance, this lack of integration represents a

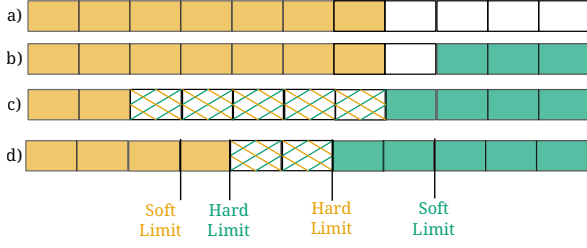


Figure 4: Different memory scenarios with two process utilizing UVM — (a) A single process, (b) Two processes with no memory oversubscription, (c) Two processes with cumulative oversubscribed memory, (d) Handling of oversubscription scenario using BoxD.

fundamental mismatch between GPU memory management and rest of the system.

The central aim of **BoxD** is to provide deterministic GPU memory to applications. Towards this, the abstraction of an application is mapped to a group of processes. The Linux kernel provides a mature and widely deployed framework for resource management in the form of control groups (cgroups). Cgroups offer native kernel mechanisms for isolating, accounting, and controlling resource consumption for resources such as CPU, system memory, and I/O at the granularity of process groups. Extending this framework to encompass GPU-managed memory is, therefore, a natural progression. Such integration would expose GPU memory as a first-class, OS specified and managed resource, enabling administrators and orchestration systems to specify and apply policies across CPUs, memory, and accelerators using a single, well-understood interface, rather than relying on fragmented, driver-specific mechanisms.

Despite the many GPU sharing mechanisms that are available, there is no mechanism to provide memory isolation for workloads that utilize unified virtual GPU memory. UVM allocated GPU memory operates within a device-specific namespace and remains globally managed without any process-aware management primitives. As GPU memory allocation hits the maximum watermark, the UVM driver initiates eviction (migration) of pages to the CPU DRAM. As shown in Figure 3, the driver uses **different lists — free, unused, and used** — to select a chunk of GPU memory for eviction. The free lists store chunks that are not currently allocated. These are organized by memory type, chunk size, and whether the chunks are zeroed or not. Root chunks unused by VA blocks (allocated but not holding any resident pages) take priority when evicting, as no data need to be migrated. The used list has root chunks used by VA blocks, likely with resident pages. These lists are objects of the per-GPU physical memory manager and not process/application specific. As a result, as the GPU memory pressure increases, the probability of evicting memory from processes that do not induce the pressure increases. Processes/applications with aggressive GPU memory access patterns can interfere with *victim* processes.

We classify execution scenarios based on the memory demands of a victim process-group (size V), an aggressor process-group (size A), the physical GPU capacity C , and the available host CPU memory H . In the *isolation* case, only the victim executes, with $V \leq C$ and $A = 0$, ensuring that the entire working set fits within the

Table 2: Runtime statistics of the victim GPU process under different content scenarios.

Case	Mean (s)	P99 (s)	Standard Deviation (s)	Evictions (GBs)	Total Faults
Isolation	11.50	11.89	0.16	0	2.26e7
No Oversubscription	13.76	13.99	0.17	0	1.58e7
Oversubscription	232.96	1053.45	148.82	12071.77	1.18e9

GPU memory. In the *no-oversubscription* case, both the victim and the aggressor execute concurrently, but their combined memory demand satisfies $V + A \leq C$; while contention for compute may arise, GPU memory pressure does not. In contrast, the *oversubscription* case arises when the combined memory demand exceeds the GPU capacity ($V + A \geq C$), yet remains within the total available device and host memory budget ($V \leq H + C - \epsilon$, $A \leq H + C - \epsilon$). In this regime, the excess demand $(V + A) - C$ must be satisfied through migration of $H \geq (V + A) - C + \epsilon$ memory to the host, triggering eviction-driven interference with default UVM. Figure 4 shows these different scenarios for using GPU memory with a representative of two processes. Figure 4(c) is a likely scenario with GPU memory oversubscription and the default UVM driver. Both processes can interfere with each others memory requirements, and the non-determinism of resident GPU memory can affect page fault occurrences and application performance.

Using a synthetic benchmark, we demonstrate the above mentioned GPU memory usage scenarios. Our setup included an NVIDIA Blackwell 4500 (32GB GPU memory), and a host with 128GB of DDR4 RAM on a PCIe 3 interface. Our synthetic benchmark (hereafter called **uvm_work**) allocated a large pool of GPU memory using `cudaMallocManaged()` and each thread of a thread pool iterated over allocated memory several times in a regular strides (128 bytes) pattern. All experiments were conducted with MPS[25]-based **Streaming Multiprocessor (SM)** [19] allocation and isolation to remove uncertainty due to scheduling. Each process was allocated 50% of the SMs. Our setup used two processes, a *victim* process and a perpetual running *aggressor* process. **The victim process’s memory requirement was 16GB, and the aggressor consumed 10GB in the no oversubscription scenario, and 20GB in the oversubscription scenario.**

Our results, with measurements collected over 100 runs in each setup, are as shown in Table 2. **Due to memory eviction pressure by the aggressor process in the oversubscription scenario, substantial increase in both the mean runtime (by more than 20×) and standard deviation (a 930× increase) were observed.** This problem becomes further exacerbated with no MPS compute partitioning. Despite the workload and access pattern remaining unchanged, exceeding GPU capacity alone is sufficient to trigger eviction–refault amplification, runaway tail latency, and severe non-determinism. These results demonstrate that with default UVM memory management, memory oversubscription transforms predictable workloads into unstable, thrashing regimes driven by global process-agnostic eviction.

Our objective in controlling GPU-managed memory allocations using our **BoxD solution are as follows:**

- Provide system control on the resident GPU memory for any process group/application.

- Provide the ability to dynamically resize allocation levels of the resident GPU memory on a per-process group basis.
- Provide a framework for process-group/application specific eviction policies.
- Integrate control of GPU memory per-process-group in existing control architectures.
- Provide a framework for making management decisions using real-time statistics on managed memory.

In the following section, we present the design and implementation details of **BoxD** to achieve these goals.

4 BoxD Design and Implementation

BoxD is designed to integrate the management of GPU Unified Virtual Memory (UVM) into the Linux cgroup subsystem. **BoxD** extends cgroups with explicit per-GPU memory controls, enabling the kernel to observe and regulate GPU-resident memory usage at the level of process groups. **BoxD** enables four specification parameters per-cgroup and per-GPU— *soft limit*, *hard limit*, *a critical limit*, and *eviction number*. The design is guided by two principles: (i) guaranteeing a minimum GPU memory allocation (the soft limit), to ensure workload stability for throughput-sensitive applications, and (ii) enforcing a strict upper bound (the hard limit) to prevent GPU memory monopolization and interference effects. Additionally, the eviction policy can be configured on a per-cgroup basis, allowing each cgroup to specify how page evictions should be handled by the driver. **BoxD** supports evicting multiple pages from a cgroup in a single operation, where the number of pages reclaimed is determined by the *eviction number*. However, aggressively reclaiming multiple pages may cause thrashing for memory-constrained cgroups. To mitigate this, we introduce a *critical limit*. If a cgroup’s memory usage falls below the critical limit, only a single page is evicted; otherwise, up to the configured *eviction number* of pages are evicted. **BoxD**’s integration with the cgroup subsystem also enables dynamic control of these specifications. With multi-GPU systems ($g_1, g_2, \dots, g_m$), **BoxD** maintains independent configurations (soft limit, hard limit, critical limit, eviction number) for each GPU, allowing policies to reflect heterogeneous device capacities and workload placement. By enforcing these limits at runtime through the cgroup interface, **BoxD** replaces opaque, driver internal eviction behavior with explicit application and/or OS-managed policies.

Figure 4(d) illustrates a simplified scenario with two applications and oversubscribed GPU memory, and **BoxD** enabled memory provisioning. With **BoxD**, GPU memory usage is constrained by the cgroup-specified soft and hard limits. Each application is guaranteed retention of its soft-limit allocation, which is protected from eviction by other cgroups, while allocations beyond this threshold remain reclaimable under memory pressure. When a process attempts to exceed its hard limit, further growth in GPU memory is prevented, forcing the workload to adapt rather than displacing the memory owned by other cgroups. This policy-driven eviction replaces implicit driver-managed contention with explicit kernel enforced boundaries, eliminating uncontrolled thrashing and non-deterministic interference, while preserving predictable co-location behavior. Ultimately, our goal is to empower the application and the operating system with the ability to dynamically reconfigure GPU memory management policies, even during active GPU kernel

execution. This functionality is foundational to the paradigm of execution state-aware computing to modulate resource allocation strategies in response to real-time changes of workloads.

4.1 Monitoring UVM-enabled Memory

An important challenge towards development of **BoxD** is the observability gap left behind by standard GPU monitoring utilities. Existing general-purpose tools [23, 26] lack the granularity and metrics to inspect the behavior inherent to UVM based managed GPU memory. Specifically, conventional tools cannot adequately capture, in real time, statistics related to on-demand page migration and faulting mechanisms that are fundamental to how UVM operates. Per-process metrics like GPU memory, CPU-pinned memory, number of page faults are not available. This limitation makes it difficult to diagnose performance issues unique to UVM-based applications, such as memory thrashing, where pages are excessively migrated between the CPU and GPU, leading to high latency and degraded performance.

As part of **BoxD**, we develop a UVM-centric ncurses-based tool—*nuvmtop*— that provides developers with actionable, real-time insights into the number of evicted bytes, number of page faults, and the GPU resident memory on per process basis. As part of *nuvmtop*, the *va_space* object is extended to include a set of performance counters, which are integrated with the UVM driver’s existing perf-based event notification mechanism for updates. Instrumentation hooks are injected into the driver’s critical execution paths, especially on the fault handling and memory allocation and eviction events. Unlike standard profiling tools that aggregate metrics globally, our modifications capture events at granularity of individual processes (via CUDA contexts), allowing for precise correlation of memory migration overheads.

4.2 BoxD Components

BoxD consists of two tightly coupled components—(i) a Linux cgroup subsystem that exposes GPU memory controls to the kernel and user space, and (ii) modifications to the *nvidia-uvmm* driver that enable cgroup-aware GPU memory allocation and eviction decisions. The cgroup subsystem is responsible for tracking the process membership, maintaining the configuration state of each cgroup, and providing a stable interface to configure memory limits. The UVM driver consumes this information to enforce the policy during GPU memory allocation and eviction. **BoxD** enforces the following invariants— (i) Once memory usage of a cgroup reaches its hard limit, on subsequent memory allocations, **BoxD** never evicts GPU memory belonging to another cgroup (ii) GPU memory usage below a cgroup’s soft limit is protected from cross-cgroup evictions, and (iii) Eviction is triggered only when the allocation fails under memory pressure. In the next two sections, we describe the implementation details of these components and their integration with the UVM driver.

4.3 The BoxD CGroup Subsystem

The key insight behind the **BoxD**’s implementation is that UVM’s eviction decisions are based on a GPU-wide view of memory pressure, without regard to process grouping or ownership. As illustrated in Figure 3, the GPU physical memory manager (PMM) maintains global eviction lists, causing eviction to occur in all processes

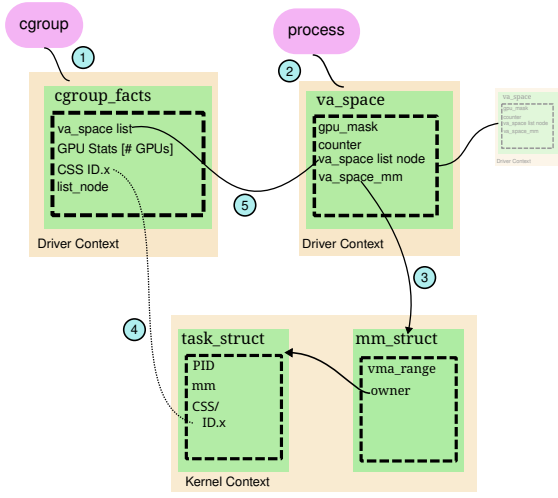


Figure 5: Linkages between information items maintained in the UVM driver and the kernel for a process's managed GPU memory.

sharing the device. BoxD modifies this behavior by explicitly making the eviction decision cgroup-aware.

A fundamental challenge arises from the mismatch between UVM's internal process abstraction and the Linux cgroup model. UVM operates primarily on the `va_space` associated with the faulting process, whereas cgroups organize processes based on their `task_struct` and PID-based membership. Bridging these two views is essential for enforcing cgroup-level GPU memory policies. With BoxD modifications, during initialization, the `nvidia-vm` module registers with the cgroup subsystem to receive notifications about cgroup lifecycle events, including creation, destruction, membership changes, and limit updates. The driver maintains an internal representation of the cgroups, tracking configured limits, current usage, and the set of processes associated with each group.

Figure 5 illustrates how BoxD bridges the abstraction of cgroups in the Linux kernel with the per-process memory structures of the NVIDIA UVM driver. The link unfolds as follows. (1) When a new cgroup is created, the BoxD cgroup subsystem initializes a corresponding `cgroup_facts` object within the UVM driver context. This structure stores the GPU accounting state, the associated CSS(Cgroup SubSystem) identifier (CSS ID.x) and maintains a list of `va_space` instances belonging to that cgroup. (2) When a process spawns and initializes CUDA, it is attached to the UVM driver, creating a new `va_space` structure in the driver context. This structure tracks GPU-specific metadata such as the `gpu_mask`, per-GPU counters, and a pointer to the process address space (`va_space_mm`). (3) The `va_space` stores a reference to the Linux `mm_struct`, allowing the driver to associate GPU-managed memory with the kernel's representation of the process address space and its virtual memory regions. (4) Through the `mm_struct`, BoxD resolves the owning `task_struct` and retrieves the corresponding CSS ID. This identifier uniquely determines the cgroup to which the process belongs. (5) Finally, the driver links the `va_space` to the appropriate

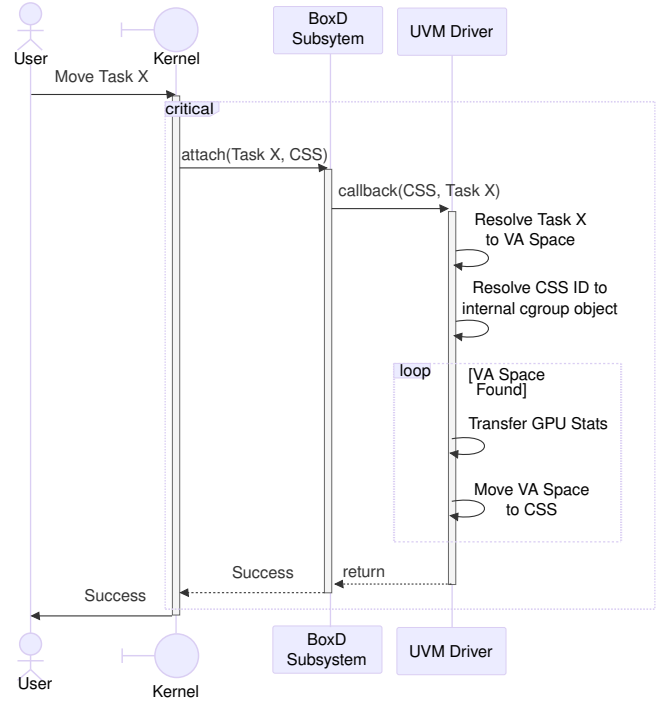


Figure 6: Sequence capturing the migration of a task to a different cgroup.

`cgroup_facts` list. From this point onward, GPU memory accounting and eviction decisions are performed with cgroup awareness, enabling BoxD to enforce per-cgroup limits and policies.

Figure 6 shows the sequence of operations when a task migrates between cgroups. Upon receiving a membership change callback, the driver updates its internal accounting state but deliberately avoids any proactive eviction or reclamation. Instead, BoxD relies on its allocation and eviction policy (Algorithm 1) to enforce limits lazily, only when memory pressure arises. This design choice ensures that BoxD remains space-efficient and avoids any unnecessary disruptions to running workloads, while still guaranteeing that memory can be reclaimed from a cgroup when required by competing processes.

Once the process is linked to the cgroup, we shift focus on how to embed the associated block information to the cgroup. A natural design alternative would be to embed GPU memory block metadata directly within the cgroup data structures. However, this approach would impose significant overhead during process termination, as reclaiming GPU memory would require traversing all the blocks associated with a cgroup, an operation that is already on the critical path of eviction. To avoid this cost, BoxD associates GPU memory blocks with the `va_space` rather than the cgroup itself, allowing block reclamation to follow the existing UVM mechanism while still enforcing cgroup-aware policy.

4.4 Memory Eviction Handling with BoxD

BoxD leaves the core chunk allocation, eviction, and deallocation mechanisms of UVM unchanged. Introduces a cgroup-aware policy

layer that governs eviction target selection without altering the underlying memory management pipeline.

4.4.1 Allocation and eviction algorithm. The key difference lies in how eviction targets are selected under memory pressure. The algorithm 1 implements a cgroup-aware allocation strategy that determines whether eviction is necessary and, if so, from which cgroup should the memory be reclaimed. The decision is based on the current GPU memory usage of the requesting cgroup relative to its configured soft and hard limits. The requesting cgroup could be (1) below its soft limit on the specific GPU, (2) above its soft limit and below critical limit, (3) just below the hard limit; When below soft limit, we evict pages from other cgroups to satisfy the demand. If the cgroup from which the pages are being evicted is below its critical limit, we only evict one page; else, we evict the number of pages as specified by the cgroup policy. Allocation proceeds optimistically as long as the requested chunk does not cause the cgroup’s usage to exceed its hard limit. In this case, the allocator first attempts to reuse a pre-allocated chunk and, failing that, allocates a new chunk without eviction, if the allocation causes the cgroup’s usage to exceed its soft limit, the cgroup becomes eligible for eviction and is added to the eviction candidate set.

4.4.2 Intra-cgroup eviction. If allocation fails, BoxD distinguishes between two cases. When the requesting cgroup has reached its hard limit, the eviction is strictly local: only chunks belonging to the same cgroup may be reclaimed. In this case, the eviction target is selected from the heaviest memory user within the cgroup, ensuring that no other cgroup is penalized. Alternative policies, such as round-robin were considered but rejected due to their interaction with high fault rates under GPU memory oversubscription.

In oversubscribed regimes, page faults occur at a higher frequency, and eviction decisions lie directly on the allocation path. Consider a cgroup in which a single process initially consumes the entire hard limit. When a second process is spawned or migrated into the same cgroup, both processes must share this fixed budget. Under a round-robin process selection policy, eviction pressure can disproportionately affect the newly introduced process, rapidly exhausting its set of work. This can lead to a pathological behavior in which the new process repeatedly faults or fails to make any progress, despite the cgroup usage being within limits. Selecting the heaviest memory consumer avoids the failure mode by biasing eviction towards the process most responsible for the memory pressure, while preserving the behavior for the lighter processes. This policy minimizes disruption within the cgroup, avoids starvation of newly admitted processes, and preserves semantics consistent with vanilla UVM behavior when a single dominant process is present. Importantly, the heaviest-process strategy provides a stable and predictable eviction rule under high fault rates without introducing additional per-process state or complex co-ordination logic in the eviction path. This behavior can be observed in our correctness evaluations in Figure 7.

It is worth situating this approach relative to classical OS page reclaim policies. Algorithms such as CLOCK approximate the principle of least-recently-used at page granularity, using recency of access as a proxy for future utility. This works well for workloads with temporal locality, where recently accessed pages are likely to be accessed again. However, under GPU memory oversubscription,

Algorithm 1 GPU Chunk Allocation with Cgroup-Aware Eviction.

```

1: Input: Requested chunk size  $s$  Soft Limit  $S_c$  and Hard Limit  $H_c$ 
2: Check:  $cgroup\_usage + s < H_c$ 
3: if True then
4:   Try to get a pre-allocated chunk
5:   if chunk found then
6:     return Success
7:   else
8:     Try to allocate new chunk (no eviction)
9:     if allocation succeeded then
10:      if  $cgroup\_usage + s > S_c$  then
11:        Add cgroup in eviction list
12:      end if
13:      return: Success
14:    end if
15:  end if
16: end if
                                ▶ At this point, allocation failed
17: if  $cgroup\_usage + s \geq H_c$  then
18:   Evict from self cgroup:
19:   Select heaviest memory user as eviction target
20: else
21:   Try evicting from cgroup exceeding soft limit
22:   if no such cgroup found then
23:     Evict from current process as last resort
24:   end if
25: end if
26: Attempt allocation again with eviction
27: if allocation succeeded then
28:   Add chunk to temporary list
29:   return: Retry Required
30: else
31:   return: Failure
32: end if

```

this assumption breaks down: the eviction–re-fault cycle causes pages to be faulted in, evicted shortly thereafter, and immediately re-demanded. In this regime, all pages appear recently used, rendering recency a poor eviction signal. More fundamentally, page-level policies are process-agnostic — they have no notion of ownership, quota, or priority across tenants. BoxD’s unit of analysis is the process and cgroup, not the individual page, reflecting the view that under multi-tenant oversubscription, fairness and isolation are policy concerns that cannot be resolved at page granularity alone.

4.4.3 Inter-cgroup eviction. When the requesting cgroup is below its hard limit, BoxD attempts to evict memory from another cgroup whose usage exceeds its soft limit. This cgroup is selected on the basis of *overdraft*, i.e. how far the cgroup is from its guaranteed memory allocation. The farthest is put up for eviction. If no such groups exist, eviction falls back to the requesting process as a last resort. This policy includes a two-tier classification of groups: (1) cgroups whose GPU memory usage exceeds its own soft limits, and (2) cgroups whose usage remains below their soft limits. Only cgroups in the first tier are eligible eviction targets for other

cgroups. The second tier cgroups are protected and may freely acquire memory from the global free list or other cgroups that have exceeded their soft limits. This tiered design ensures BoxD is space-conserving: in the absence of competing workload, a cgroup could grow to its hard limit without interference.

The driver maintains a usage counter per-cgroup s_c , which is updated on all allocation and deallocation paths. Upon allocation, s_c increases with the size of the group. UVM deallocates GPU memory exclusively in fixed-size chunks of 2MB. Consequently, on deallocation s_c is decremented by the size of the evicted chunk. To avoid repeatedly selecting the same cgroup for eviction, BoxD tracks all cgroups exceeding their soft limits in a circular eviction queue, ensuring fairness among eligible eviction candidates.

4.5 Multi-GPU Awareness and Accounting

The primary challenge in supporting multi-GPU systems is establishing a consistent device identity that is shared between BoxD’s cgroup subsystem and the `nvidia-uvm` driver. Both components must agree on a stable GPU identifier in-order to enforce per-device memory limits and perform accounting. UVM internally assigns GPU identifiers based on the system’s PCIe topology. Discrete GPU are enumerated in PCIe order, with the first device assigned GPU ID 0. The driver supports up to 32 physical GPU, and each physical device may expose up to 8 sub-devices through NVIDIA’s Multi-Instance GPU (MIG [24]) mechanism. As a result, GPU identifiers are allocated in contiguous blocks of eight: if a physical GPU is assigned processor ID k , its sub-devices occupy the range $[8k, 8k + 7]$, and the next physical GPU is assigned $8(k + 1)$.

The BoxD cgroup subsystem explicitly mirrors this convention. During initialization, GPU devices are discovered using standard Linux kernel interfaces to iterate over PCIe-attached devices, and GPU identifiers are assigned consistent with the UVM numbering scheme. This ensures that the cgroup subsystem and the driver operate on a shared and unambiguous notion of GPU identity, without introducing additional translation layers. For each cgroup, the `nvidia-uvm` module will maintain a per-GPU accounting structure. This structure records independently for each GPU (1) the current memory usage of the cgroup, (2) the identity of the heaviest memory consuming process of the cgroup, (3) the configured soft and hard limits. All accounting and enforcement decisions are therefore scoped to the specific GPU.

On the eviction path, the faulting `va_block` explicitly encodes the GPU on which the memory resides. BoxD uses this information to ensure that the eviction targets are selected exclusively from the same GPU as the allocation request. Both eviction and subsequent accounting updates are performed on a per-GPU basis, preventing cross-device interference and ensuring that limits on one GPU do not affect memory usage on another.

5 Empirical Evaluation

This section presents an empirical evaluation of BoxD from the correctness and applicability perspectives. We first verify the correctness of BoxD and its ability to adaptively change and enforce memory limits. We demonstrate the utility of BoxD through comparisons with the vanilla UVM driver. The experimental setup was made up of Intel Xeon Gold 6142 CPU, with 128GB DDR4 RAM, and

two GPUs—a NVIDIA RTX A5000 (24GB) and an NVIDIA Blackwell 4500 (32GB). [All our experiments are performed on the NVIDIA Blackwell 4500 \(32GB\) unless mentioned otherwise.](#)

Our workloads consist of two applications: (i) **uvm_work**, a custom benchmarking tool for UVM managed memory, and (ii) **cu-Graph**[21]-based graph processing server. The graph-processing server initializes a global memory pool backed by a UVM managed memory. The server maintains a thread pool, each of which is associated with a unique CUDA stream and a thread-specific memory resource derived from the global memory pool. We service each request through a thread-local execution pattern, wherein each worker thread lazily has private resource instances. This instance includes a unique CUDA stream and a thread-exclusive arena memory resource derived from the global pool. This separates the shared, read-only topology of the read-in graph.

The accesses in global memory are encapsulated by a prefetch adaptor provided by RMM[27]. The graph reading is done from Parquet formatted edge-list via `cuDF.read_parquet`[20], after which graph analytics are executed using cuGraph’s GPU optimized algorithm implementation. Such memory access patterns have been advocated in the industry [10]. We utilize Barabási-Albert (BA) graphs [3] as a controlled structural stress test. The BA model produces scale-free graphs with a power-law degree distribution and high-degree hubs.

Locust [16] is used to generate load via a trace-driven log of HTTP requests, to enable deterministic replay of load. Each trace consists of a sequence of graph operations drawn from a predefined task set — breadth-first search (BFS), community detection (Louvain), structural analytics, and weakly connected components (WCC). The selection of operations follows a weighted distribution, allowing us to model different workload intensities.

5.1 Correctness

We demonstrate the correctness of BoxD by validating both its GPU memory-limit enforcement and its integration with the cgroup abstraction in Linux. Correctness is demonstrated along two axes: (i) adherence to configured memory limits, and (ii) correct behavior under dynamic cgroup operations, including process migration and multi-GPU execution.

5.1.1 Adhering to Hard Limits. We first evaluate BoxD under a single-cgroup configuration that contains multiple concurrent processes. In this regime, soft limits are irrelevant, as they are designed to arbitrate cross-cgroup contention. Since all processes belong to the same cgroup, the hard limit is the sole governing constraint on GPU memory residency. The workload consists of three processes executing `uvm_work`: (1) P1: 12 GB, 1 iteration (2) P2: 8 GB, 7 iterations (3) P3: 6 GB, 40 iterations

We execute this workload under progressively tighter hard limits that span three regimes: (1) limit greater than cumulative demand, (2) limit equal to cumulative demand, and (3) limit strictly below cumulative demand; Across all configurations, Figure 7 shows that the aggregate residency of the GPU never exceeds the configured hard limit. Enforcement is strict and immediate: once the limit is reached, additional allocations trigger targeted reclamation within the cgroup. As the hard limit tightens below total demand, memory pressure increases, and the eviction frequency rises sharply.

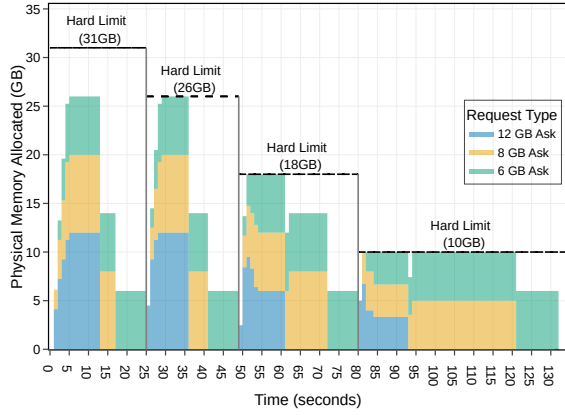


Figure 7: GPU resident memory of processes with varying hard limits of corresponding cgroup.

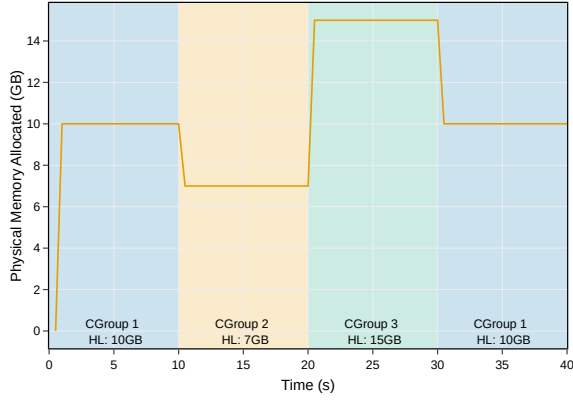


Figure 8: Resident GPU memory of a process when moving across cgroup boundaries with different hard limits.

However, BoxD prevents starvation and pathological imbalance. Reclamation preferentially targets the largest resident process, ensuring proportional pressure rather than arbitrary displacement. When a process terminates, its freed memory is immediately absorbed by the remaining processes, demonstrating responsive and dynamic redistribution within the cgroup boundary.

In particular, P1 (12 GB, single iteration) exhibits a nearly invariant runtime across all hard-limit configurations—even under constrained settings. This stability arises from its single-pass, non-reusing access pattern – evicted pages are rarely revisited, limiting eviction refault amplification. In contrast, P2 and P3 repeatedly traverse their working sets. Under tighter limits, their iterative access patterns induce sustained eviction–refault cycles, increasing memory churn and degrading performance. These results demonstrate that BoxD enforces hard limits precisely, maintains fairness, and prevents starvation, even under sustained intra-cgroup contention.

5.1.2 Process migration across cgroups. In the Linux cgroup subsystem, a process can migrate between cgroups at runtime. Such migration dynamically changes the resource controls applied to the

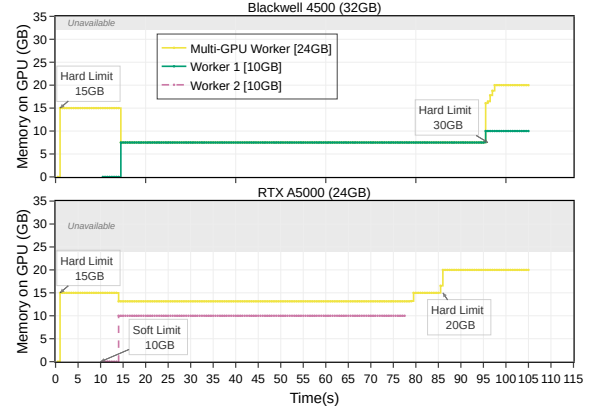


Figure 9: Resident GPU memory over time with heterogeneous GPUs under varying cgroup limits. Workers in the same cgroup share the same line style.

process, and the process subsequently operates under the resource limits and policies of the destination cgroup. The BoxD cgroup controller also supports such migration across cgroup and updates the limits dynamically. Figure 8 presents such an experiment, where `uvm_work` is configured to consume 20GB of memory on the GPU. It is cycled through three different cgroups, each having different hard limits. We observe that BoxD correctly enforces the updated limits across the process movement.

5.1.3 Multi-GPU setups and memory limits. To affirm how BoxD’s cgroup-enforced memory limits interact with heterogeneous GPU capacities, we examine the temporal evolution of per-process GPU resident memory under a mixed workload. In this experiment, multiple workers execute concurrently across both available GPUs, while we vary both the soft and hard limits. In this experiment, we launch three `uvm_work` instances. At first a multi-GPU instance is launched, which has a 24GB memory ask and is perpetually running. It is launched in a cgroup with 15GB as the hard limit for both GPUs. After 10s we simultaneously launch two single-GPU worker instances with 10GB ask and 2 iterations over them in a regular access fashion; these instances are launch on different GPUs, albeit the first worker is launched in the same cgroup as the first multi-GPU worker, the latter being launched in its own cgroup with 10GB soft and hard limit. Figure 9 illustrates how the changing limits shape the allocation behavior over time, highlighting the differences in memory utilization patterns between devices and the interactions between workers in different cgroups. The shaded region represents unavailable hardware memory.

As shown in Figure 9, when the hard limits are applied, resident memory is capped by the specific hard limit enforced for that GPU for that cgroup. This effect is particularly visible for the multi-GPU worker, whose resident memory adapts differently depending on the underlying device hard limit. We also see the effect of reserving memory using the soft limits. The second single-GPU worker finishes relatively earlier than its counterpart on the better GPU. The second worker had its entire ask memory reserved by the soft limit for its entire memory ask. The first single-GPU worker is launched

Table 3: Latency and page fault statistics for victim process with varying working set size of co-located process. M is the per-node attachment factor of a graph.

M	UVM Variant	Median (s)	P99 (s)	Total Page Faults
10	BoxD	21.67	34.81	1,030
	Vanilla	22.42	49.97	2,422,032
12	BoxD	21.77	36.59	1,027
	Vanilla	33.79	53.07	11,656,460
15	BoxD	23.79	37.85	1026
	Vanilla	39.80	62.94	21,650,905
17	BoxD	24.74	40.56	1033
	Vanilla	45.03	75.51	32,034,590

in the same cgroup as the multi-GPU worker, this causes them to share the 15GB hard limit and stabilize around 7.5GB. Figure 7 shows that this should be the expected behavior.

These observations naturally raise a key question: to what extent does interference become performance-limiting as the memory footprint of an application continues to grow? In the following subsection, we investigate this question by quantifying the scale of interference under increasing memory pressure.

5.2 Interference Under Oversubscription

In this section, we try to evaluate performance under different scenarios of memory subscription and contention.

5.2.1 Effect of increasing memory oversubscription. To quantify how interference scales under increasing GPU memory pressure, we evaluated a bully-victim workload configuration in which one application progressively increases its memory footprint while another application’s workload remains fixed. Table 3 reports the impact of this interference on the victim process, comparing BoxD with the vanilla `nvidia-uvdm` driver.

In this experiment, two `cuGraph` servers execute concurrently on the same GPU in a closed-loop request environment. Each server is initialized with a BA graph containing $N = 5 \times 10^6$ nodes. The victim server operates on a fixed graph, while the bully server increases its effective working set by varying the attachment parameter M , which controls the graph’s edge density and consequently the GPU memory footprint. Increasing M therefore induces progressively higher levels of memory oversubscription and eviction pressure. Both servers face the same number of users and the same type of request to run the Louvain algorithm on the graph.

For each configuration, we measure the median and tail (P99) request latency of the victim server, along with the total number of GPU page faults incurred during execution. Under the vanilla driver, increasing oversubscription leads to a dramatic increase in page fault activity, increasing from approximately 2.4 million page faults at $M = 10$ to more than 32 million faults at $M = 17$. This increase directly translates into higher median latency and severe tail latency amplification, with median latency doubling and P99 latency increasing by 1.5 $\times$ across the tested range. In contrast BoxD effectively bounds interference. Across all configurations,

Table 4: Runtime statistics of the victim GPU process under different contention scenarios.

Case	Mean (s)	P99 (s)	Standard Deviation (s)	Evictions (GBs)	Total Faults
Isolation	11.50	11.89	0.16	0	2.26e7
No Oversubscription	13.76	13.99	0.17	0	1.58e7
Oversubscription	232.96	1053.45	148.82	12071.77	1.18e9
BoxD with No Oversubscription	13.36	13.56	0.16	0	1.55e7
BoxD with Oversubscription	16.02	16.53	0.5	0	2.18e7

the total number of page faults experienced by the victim remains nearly constant at approximately 1,000, independent of the bully’s memory footprint. As a result, both the median and the tail latencies remain stable, with only modest increases attributable to residual contention. Even at the highest level of oversubscription, BoxD reduces P99 latency by **nearly half** compared to the vanilla driver.

These results demonstrate that without BoxD’s control mechanism, interference grows superlinearly with memory oversubscription due to uncontrolled eviction and re-faulting. By enforcing cgroup-aware limits and restricting cross-group eviction, BoxD prevents this amplification effect, ensuring that interference remains bounded even as competing workloads increase their demand.

5.2.2 Runtime Stability Under Contention. Complementing the above analysis, we next examine how the same interference manifests itself in overall runtime behavior using the simplified `uvdm_work` workload. This allows us to isolate the performance consequences of GPU memory contention from application-specific execution variability. Table 4 quantifies how GPU memory interference translates into end-to-end runtime behavior and completes the partial results introduced in Table 2 by including BoxD enabled configurations. **The process does not experience any evictions under BoxD, as its entire working set remains within its soft limit.** In isolation, the victim exhibits low variance table execution. Under vanilla UVM, even in the absence of oversubscription, runtime increases modestly due to background contention, while with oversubscription the Mean latency increases up to 20 $\times$ and the variability rises substantially. With BoxD enabled, runtime behavior closely tracks the corresponding vanilla baselines in the absence of oversubscription, indicating that BoxD introduces no measurable overhead under non-contended conditions. More importantly, under oversubscription, BoxD restores near-isolated performance: **the mean runtime increases by only $\approx 16\%$ relative to no-oversubscription case, a stark improvement over 20 $\times$ inflation observed under the vanilla UVM.** and variability remains low. This stability directly reflects BoxD’s ability to bound cross-cgroup interference by preventing uncontrolled eviction and page-fault amplification, even when competing workloads aggressively increase memory demand.

5.2.3 Impact of Co-located Processes and Soft Limits. To further understand the impact of interference on completion time as the number of concurrently executing processes increases, we vary both the number of co-located processes and the soft limit assigned to the victim process. The experiment was conducted on the RTX A5000 (24GB GPU memory). Each process was allocated 25% compute partition via MPS and executed the `uvdm_work` benchmark with

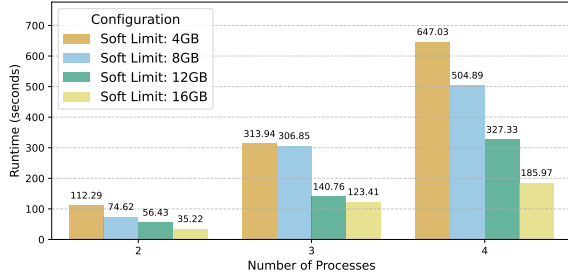


Figure 10: Effect of varying interference on a victim process.

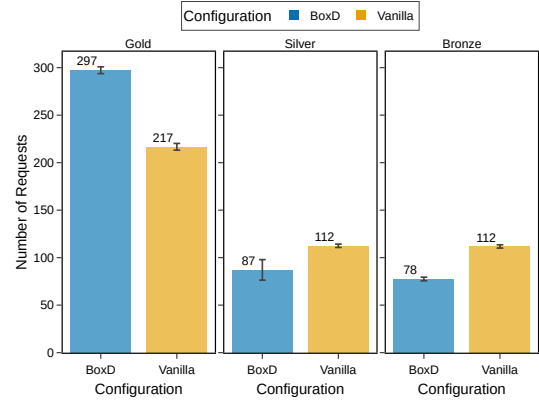
a 16GB working set and 25 iterations over the demanded memory. As shown in Figure 10, we observe that execution time of the victim process increases with the number of co-located processes (interference) and decreases as the soft limit assigned to the process increases. For example for the 12GB soft limit, increasing interference results in execution time increase from 56s to 140s to 320s, with 1, 2 and 3 interference inducing processes (total processes are reported as 2, 3, and 4, respectively). Smaller values of soft limits amplify this effect further. Not only is a larger fraction of the working set repeatedly evicted and faulted back into GPU memory, but page fault handling contention also increase with interference. With the vanilla driver, this experiment was intractable—the victim process’s completion time was ~ 6 hours with 4 co-located processes. Further, note that if the victim process’s GPU memory up to its soft limit was pre-allocated, the impact on its performance was very small. For example, in the setup of Figure 10, if the victim process configured with a soft limit of 12GB, had 12 GB pre-allocated before the interference commenced, then the measured run times were 62 seconds and 72 seconds, with 3 and 4 c-located processes, respectively. While BoxD was able to limit the page-fault contention due to the enforcement of the soft limit, the vanilla driver had not such control and led to uncontrolled eviction and fault handling.

5.3 Static Memory Differentiation

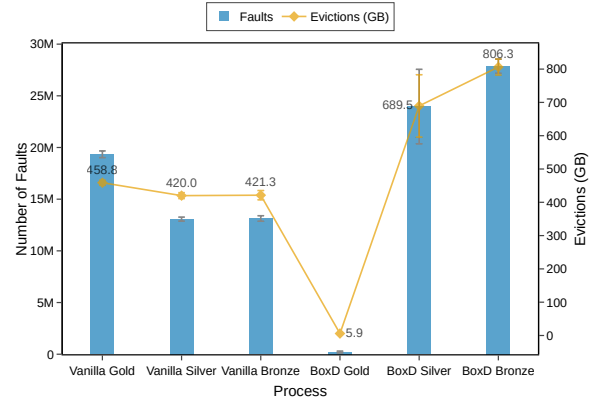
We evaluate BoxD under a statically provisioned multi-tenant setup. Three independent service instances are launched within distinct cgroups, each assigned a fixed GPU memory budget and a bounded compute share. This configuration emulates a tiered provisioning model, similar to the Gold, Silver, and Bronze service classes in production systems. Each service is launched using a fixed hard limit enforced via BoxD, with no dynamic resizing during execution. GPU compute share is controlled via an active thread percentage variable provided by MPS, ensuring that both memory and compute resources are statically partitioned. The three services are configured as follows: (i) **Gold**: 15 GB soft budget (32 GB hard limit), 50% MPS (ii) **Silver**: 9 GB soft budget (32 GB hard limit), 25% MPS (iii) **Bronze**: 8 GB soft budget (32 GB hard limit), 25% MPS

A Locust workload generator issues concurrent queries to all three services for a fixed duration. We compare vanilla UVM with BoxD under identical workload and provisioning configurations.

Figure 11a reports the number of completed requests across the three service tiers. Under BoxD, the Gold, Silver, and Bronze tiers complete 297, 81, and 78 requests respectively. In contrast, under



(a) Completed Requests over a period of 10mins.



(b) Eviction statistics for every tier.

Figure 11: Non-deterministic GPU memory residency and eviction amplification with oversubscribed UVM memory.

vanilla UVM, the corresponding numbers are 217 (Gold), 112 (Silver), and 112 (Bronze). These are the mean number of completed requests over 10 runs. The ordering under vanilla UVM does not reflect the intended provisioning hierarchy. Bronze achieves performance comparable to or exceeding Silver. In contrast, under BoxD, throughput strictly follows the provisioning hierarchy: Gold achieves the highest throughput, followed by Silver and Bronze.

Under vanilla UVM, the ordering does not cleanly reflect the intended provisioning hierarchy. Although Gold has the highest compute share assigned, Bronze achieves throughput comparable to or exceeding Silver. Notably, Gold still outperforms the other tiers in vanilla. CUDA_MPS_ACTIVE_THREAD_PERCENTAGE is partly the cause, which grants it a greater share of GPU compute resources. With greater compute share, Gold can generate page faults at a higher rate, and thus re-fault evicted pages more aggressively, allowing it to make forward progress despite global eviction pressure. However, this behavior does not constitute isolation. Figure 11b shows that under vanilla UVM all tiers incur hundreds of gigabytes of eviction traffic and tens of millions of page faults. Memory churn remains uniformly high across tiers, indicating that eviction decisions are global and policy-blind. Gold’s relative advantage stems

from its ability to refault more quickly—not from protected residency. With BoxD enabled, the mechanism changes fundamentally. The Gold-tier process incurs only 5.9 GB of eviction and negligible faults, while Silver and Bronze absorb progressively greater memory pressure, with Bronze absorbing a massive 806.3 GB of cumulative eviction. Because BoxD localizes eviction within cgroup boundaries, Gold retains its working set rather than competing via fault-rate dominance. As a result, throughput aligns with provisioning intent: compute share and memory budget translate directly into stable performance.

6 Related Work

Prior work on GPU Unified Virtual Memory (UVM) broadly falls into three categories: (1) characterization and performance analysis, (2) migration and prefetch optimization, and (3) multi-tenant resource management.

Characterization of UVM. Several studies provide fine-grained empirical analyses of UVM demand paging, quantifying fault latency, migration cost, and residency [1]. These works expose the performance implications of UVM but do not introduce mechanisms to control eviction behavior under multi-tenant contention.

Migration and Prefetch Optimization. A substantial body of work seeks to improve UVM performance by optimizing migration mechanisms or predicting access patterns. Choi et al. [4] reduce eviction pressure in multi-GPU systems through differentiated migration paths. DeepUM [12] and SUV [2] leverage learning or static analysis to guide prefetching and residency decisions. DREAM [18], Forest [14], and EarlyAdaptor [8] incorporate hardware-driven or device-based strategies to improve paging efficiency.

These approaches optimize the *mechanism* of migration—reducing latency or improving placement—but assume cooperative or single-application contexts. They do not address policy-level control over eviction between concurrently executing tenants.

Runtime and Cluster-Level GPU Sharing. Several systems tackle GPU sharing at the runtime or cluster level. PILOT proposes dynamic management of UVM memory footprint in multi-tenant environments, adapting application behavior to mitigate contention. MuxFlow [33] and D-Stack [5] focus on safe and efficient GPU sharing in production deep learning clusters. Fractional GPUs (FGPUs) [11] and subsequent works explore software-based reservation of compute and memory bandwidth resources. More recent work on GPU multitasking for LLM serving [31] explores spatial-temporal sharing strategies for inference workloads.

These systems primarily regulate compute allocation, scheduling, or application-level footprint adjustment. However, they do not modify UVM’s eviction policy or provide OS-level enforcement of managed memory residency. Memory contention remains governed by the driver’s global reclamation mechanism.

Bandwidth and Cache Isolation. Orthogonal efforts such as SGDR [32] regulate intra-GPU memory bandwidth and cache interference. Although effective in controlling SM-level contention, these techniques do not govern managed memory oversubscription or UVM eviction behavior.

7 Discussions and Future Work

With growth of memory heavy applications usage of UVM is bound to grow, especially with Linux integrating HMM and providing an unified approach to manage memory. An important design option is for the host OS managing heterogeneous memory, e.g., the Heterogeneous Memory Management (HMM) feature of Linux. As work-in-progress, we modified the memory management sequence of vLLM (a LLM serving engine) [13] to enabled uvm-managed allocation of memory pools via caching allocators of PyTorch [30]. With this modification, new possibilities of symbiotic memory management between the application-layer and the OS open up for exploration. For example, BoxD uses the application layer hints for prefetching, access patterns, and offloading as hints for efficient driver actions.

As multi-tenant GPU deployments scale toward large clusters, node-local enforcement mechanisms like BoxD must seamlessly integrate with global orchestration frameworks. BoxD’s use of standard Linux cgroups provides a natural abstraction for cluster-level deployment and orchestration (e.g., Docker + Kubernetes). BoxD’s cgroup interface readily provides an interface to dynamically modulate memory limits based on fleet-wide telemetry. Future work involves integrating BoxD with such cgroup-based deployment platforms and controllers.

8 Conclusion

Unified Virtual Memory simplifies GPU programming and enables oversubscription, but introduces a fundamental policy gap: eviction decisions are global and process-agnostic, operating outside the operating system’s resource governance framework. As our motivating experiments demonstrate, exceeding GPU capacity alone is sufficient to induce eviction-refault amplification, hundreds of gigabytes to terabytes of memory churn, and severe tail latency inflation even for identical workloads.

We presented BoxD, a cgroup-backed mechanism that integrates GPU-managed memory into the Linux resource control subsystem. Rather than modifying UVM’s allocation or migration mechanisms, BoxD introduces policy-level control over eviction by making memory reclamation cgroup-aware. This approach preserves UVM’s existing infrastructure while restoring deterministic hard and soft limit enforcement under oversubscription.

Our evaluation shows that BoxD eliminates uncontrolled global churn, aligns memory residency with provisioning intent, and restores predictable throughput across statically provisioned service tiers. Under contention, higher-priority workloads retain their working sets instead of competing via fault-rate dominance, preventing runaway tail latency and non-deterministic behavior.

These results suggest that GPU-managed memory must be treated as a first-class operating system resource. As GPU sharing becomes central to modern multi-tenant and LLM-serving systems, policy-aware memory governance is essential for predictable, isolation-preserving execution.

References

- [1] Tyler Allen, Bennett Cooper, and Rong Ge. 2024. Fine-grain quantitative analysis of demand paging in unified virtual memory. *ACM Transactions on Architecture and Code Optimization*, 21, 1, Article 14, (Jan. 2024), 24 pages. doi:10.1145/3632953.

- [2] Pratheek B, Guilherme Cox, Jan Vesely, and Arkaprava Basu. 2024. Suv: static analysis guided unified virtual memory. In *2024 57th IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 293–308. doi:10.1109/MICRO61859.2024.00030.
- [3] Albert-László Barabási and Réka Albert. 1999. Emergence of scaling in random networks. *science*, 286, 5439, 509–512.
- [4] Sangjin Choi, Taeksoo Kim, Jinwoo Jeong, Rachata Ausavarungnirun, Myeongjae Jeon, Youngjin Kwon, and Jeongseob Ahn. 2022. Memory harvesting in Multi-GPU systems with hierarchical unified virtual memory. In *2022 USENIX Annual Technical Conference (USENIX ATC 22)*, (July 2022), 625–638. ISBN: 978-1-939133-29-66. <https://www.usenix.org/conference/atc22/presentation/choi-sangjin>.
- [5] dstack GmbH. 2026. AI container orchestration for AI teams. [Online; accessed 14. Feb. 2026]. (Feb. 2026). Retrieved Feb. 14, 2026 from <https://dstack.ai>.
- [6] Debashis Ganguly, Ziyu Zhang, Jun Yang, and Rami Melhem. 2020. Adaptive page migration for irregular data-intensive applications under gpu memory oversubscription. In *2020 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, 451–461. doi:10.1109/IPDPS47924.2020.00054.
- [7] Guin Gilman, Samuel S. Ogden, Tian Guo, and Robert J. Walls. 2021. Demystifying the placement policies of the nvidia gpu thread block scheduler for concurrent kernels. *SIGMETRICS Performance Evaluation Review*, 48, 3, (Mar. 2021), 81–88. doi:10.1145/3453953.3453972.
- [8] Seokjin Go, Hyunwuk Lee, Junsung Kim, Jiwon Lee, Myung Kuk Yoon, and Won Woo Ro. 2023. Early-adaptor: an adaptive framework for proactive uvm memory management. In *2023 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, 248–258. doi:10.1109/ISPASS57527.2023.00032.
- [9] 2019. Heterogeneous Memory Management (HMM) — The Linux Kernel documentation. [Online; accessed 11. Oct. 2025]. (Mar. 2019). Retrieved Oct. 11, 2025 from <https://www.kernel.org/doc/html/v5.0/vm/hmm.html>.
- [10] 2026. Introducing UVM for larger than VRAM data on the Polars GPU engine. [Online; accessed 21. Jan. 2026]. (Jan. 2026). Retrieved Jan. 21, 2026 from <https://pola.rs/posts/uvm-larger-than-ram-gpu>.
- [11] Saksham Jain, Iljoo Baek, Shige Wang, and Ragunathan Rajkumar. 2019. Fractional gpus: software-based compute and memory bandwidth reservation for gpus. In *2019 IEEE Real-Time and Embedded Technology and Applications Symposium (RTAS)*, 29–41. doi:10.1109/RTAS.2019.00011.
- [12] Jaehoon Jung, Jinpyo Kim, and Jaejin Lee. 2023. Deepum: tensor migration and prefetching in unified memory. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (ASPLOS 2023)*. Association for Computing Machinery, New York, NY, USA, 207–221. ISBN: 9781450399166. doi:10.1145/3575693.3575736.
- [13] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient memory management for large language model serving with pagedattention. In *In Proceedings of the 29th ACM Symposium on Operating Systems Principles (SOSP '23)*. New York, NY, USA, 611–626. ISBN: 9798400702297. doi:10.1145/360006.3613165.
- [14] Mao Lin, Yuan Feng, Guilherme Cox, and Hyeran Jeon. 2025. Forest: access-aware gpu uvm management. In *Proceedings of the 52nd Annual International Symposium on Computer Architecture (ISCA '25)*. Association for Computing Machinery, 137–152. ISBN: 9798400712616. doi:10.1145/3695053.3731047.
- [15] Zejia Lin, Hongxin Xu, Guanyi Chen, Zhiguang Chen, Yutong Lu, and Xianwei Zhang. 2025. Boosting LLM Serving through Spatial-Temporal GPU Resource Sharing. *arXiv*, (Apr. 2025). eprint: 2504.19516. doi:10.48550/arXiv.2504.19516.
- [16] 2026. Locust.io. [Online; accessed 14. Feb. 2026]. (Feb. 2026). Retrieved Feb. 14, 2026 from <https://locust.io>.
- [17] Praneel Madabushini. 2025. Optimizing GPU Utilization for AI Workloads on AWS EKS. *World Journal of Advanced Research and Reviews*, 26, 1, 1955–1963. doi:10.30574/wjarr.2025.26.1.1233.
- [18] Nurlan Nazaraliyev, Elahesh Sadredini, and Nael Abu-Ghazaleh. 2025. Dream: device-driven efficient access to virtual memory. In *Proceedings of the 39th ACM International Conference on Supercomputing (ICS '25)*, 1190–1205. ISBN: 9798400715372. doi:10.1145/3721145.3725748.
- [19] NVIDIA Corporation. 2025. *CUDA C++ Programming Guide*. Accessed: 2026-05-29. <https://docs.nvidia.com/cuda/cuda-c-programming-guide/>.
- [20] NVIDIA Corporation. 2024. cuDF Pandas. [Online; accessed 15. Oct. 2025]. (Jan. 2024). Retrieved Oct. 15, 2025 from <https://rapids.ai/cudf-pandas>.
- [21] NVIDIA Corporation. 2025. cuGraph Introduction — cugraph-docs 25.10.00 documentation. [Online; accessed 15. Oct. 2025]. (Oct. 2025). Retrieved Oct. 15, 2025 from https://docs.rapids.ai/api/cugraph/stable/basics/cugraph_intro.
- [22] NVIDIA Corporation. 2026. Enhancing distributed inference performance with the nvidia inference transfer library. Accessed: 2026-05-29. <https://developer.nvidia.com/blog/enhancing-distributed-inference-performance-with-the-nvidia-inference-transfer-library/>.
- [23] NVIDIA Corporation. 2026. Nsight Compute CLI — NsightCompute 13.1 documentation. [Online; accessed 14. Feb. 2026]. (Jan. 2026). Retrieved Feb. 14, 2026 from <https://docs.nvidia.com/nsight-compute/NsightComputeCli/index.html>.
- [24] NVIDIA Corporation. 2025. NVIDIA Multi-Instance GPU (MIG). (Oct. 2025). Retrieved Oct. 10, 2025 from <https://www.nvidia.com/en-us/technologies/multi-instance-gpu>.
- [25] NVIDIA Corporation. 2025. NVIDIA Multi-Process Service (MPS). (Oct. 2025). Retrieved Oct. 10, 2025 from <https://docs.nvidia.com/deploy/mps/index.html>.
- [26] NVIDIA Corporation. 2025. NVIDIA System Management Interface (nvidia-smi). [Online; accessed 14. Feb. 2026]. (Dec. 2025). Retrieved Feb. 14, 2026 from <https://docs.nvidia.com/deploy/nvidia-smi/index.html>.
- [27] NVIDIA Corporation. 2026. RAPIDS Memory Manager (RMM). Accessed: 2026-05-22. <https://github.com/rapidsai/rmm>.
- [28] NVIDIA Corporation. 2022. Unified Memory for CUDA Beginners | NVIDIA Technical Blog. [Online; accessed 13. Feb. 2026]. (Aug. 2022). Retrieved Feb. 13, 2026 from <https://developer.nvidia.com/blog/unified-memory-cuda-beginners>.
- [29] NVIDIA Dynamo Team. 2026. NVIDIA Inference Xfer Library (NIXL). <https://github.com/ai-dynamo/nixl>. Accessed: 2026-06-12. (2026).
- [30] Adam Paszke et al. 2019. PyTorch: An Imperative Style, High-Performance Deep Learning Library. *arXiv*, (Dec. 2019). eprint: 1912.01703. doi:10.48550/arXiv.1912.01703.
- [31] Jiarong Xing, Yifan Qiao, Simon Mo, Xingqi Cui, Gur-Eyal Sela, Yang Zhou, Joseph Gonzalez, and Ion Stoica. 2025. Towards Efficient and Practical GPU Multitasking in the Era of LLM. *arXiv*, (Aug. 2025). eprint: 2508.08448. doi:10.48550/arXiv.2508.08448.
- [32] Yongkang Zhang et al. 2025. Sgdr: software-defined dynamic resource control for concurrent dnn inference on nvidia gpus. In *Proceedings of the 30th ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming (PPoPP '25)*, 267–281. ISBN: 9798400714436. doi:10.1145/3710848.3710863.
- [33] Yihao Zhao, Xin Liu, Shufan Liu, Xiang Li, Yibo Zhu, Gang Huang, Xuanzhe Liu, and Xin Jin. 2023. MuxFlow: Efficient and Safe GPU Sharing in Large-Scale Production Deep Learning Clusters. *arXiv*, (Mar. 2023). eprint: 2303.13803. doi:10.48550/arXiv.2303.13803.